

PLFA in Lean

Daniel

Contents

Contents	i
1 Part 1	1
1.1 Naturals	1

1 Part 1

1.1 Naturals

Most are familiar with the natural numbers:

0
1
2
3
...

and so on. The set of natural numbers $\mathbb{N}$ is infinite and we say that 0, 1, 2, 3, and are **values** of type $\mathbb{N}$ (otherwise indicated with: $0 : \mathbb{N}$, $1 : \mathbb{N}$, etc.).

Although there are infinitely many naturals, yet we can write down its definition in just a few lines. Here it is as a pair of inference rules:

```
-----  
zero :  $\mathbb{N}$   
  
m :  $\mathbb{N}$   
-----  
suc m :  $\mathbb{N}$ 
```

And here is the definition in Lean:

```
inductive  $\mathbb{N}$  : Type where  
  | zero :  $\mathbb{N}$   
  | suc  :  $\mathbb{N} \rightarrow \mathbb{N}$ 
```

Here $\mathbb{N}$ is the name of the datatype we are defining, and zero and suc (short for successor) are the **constructors** of the datatype.

Both definitions above tell us two things:

1. base case: zero is a natural number

2. inductive case: if m is a natural number, then $\text{succ } m$ is also a natural number.

Further, these two rules give the **only** ways of creating natural numbers. Hence, the possible natural numbers are:

```
.zero
.succ .zero
.succ (.succ .zero)
...
```

We write 0 as the shorthand for the zero constructor and 1 as shorthand for $\text{succ } 0$, the successor of zero, that is the number that comes after 0 .

Exercise: seven (practice)

Write out 7 in longhand.

potential sol:

```
def seven : ℕ :=
  .succ $ .succ $ .succ $ .succ $ .succ $ .succ $ .succ .zero
```

Unpacking the inference rules

Let's unpack the Lean definition. the keyword `inductive` tells us this is an inductive definition, that is, we are defining a new datatype with constructors. The phrase:

$\mathbb{N} : \text{Type}$

tells us that $\mathbb{N}$ is the name of the new data, type and that it is a Type, which is the way in Lean of saying that it is a type.

The keyword `where` is the name of the new datatype, and that is a Type, which is the way in Lean of saying that it is a type. The keyword `where` separates the declaration of the data from the declaration of its constructors. Each constructor is declared on a separate line, which is indented to indicate that it belongs to the corresponding data declaration. The lines:

```
zero : ℕ
succ  : ℕ -> ℕ
```

give *signatures* specifying the types of the constructors `zero` and `suc`. They tell us `zero` is a natural number and that `suc` takes a natural number as an argument and returns a natural number.

You may have noticed that $\mathbb{N}$ and $\rightarrow$ don't appear on your keyboard. They are symbols in *unicode*. At the end of each chapter is a list of all unicode symbols introduced in the chapter, including instruction on how to insert them into the editor (assuming you're using zed or intellij).

Lean4 Natural Literal Syntax

Lean4 already defines its own inductive type for the natural numbers and also supports integer literal syntax. We can hook into that syntax and relate it to our own $\mathbb{N}$ type with the following conversion function and typeclass instance implementation:

```
def convert : _root_.Nat ->  $\mathbb{N}$ 
| _root_.Nat.zero   => .zero
| _root_.Nat.succ n => .suc (convert n)

instance (n : _root_.Nat) : OfNat  $\mathbb{N}$  n where
  ofNat := convert n
```

Operations on naturals are recursive functions

Now that we have the natural numbers, what can we do with them? For instance, can we define arithmetic operations such as addition and multiplication?

It turns out, each of these can be described as *recursive* functions. Here is the definition of addition for the $\mathbb{N}$ type in lean4:

```
def plus :  $\mathbb{N}$  ->  $\mathbb{N}$  ->  $\mathbb{N}$ 
| .zero , n      => n
| (.suc m) , n => .suc (plus m n)

syntax:65 (priority := high) term:66 " + " term:65 : term

macro_rules
| `($a + $b) => `(plus $a $b)
```

The first line of `plus` gives its type: $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, which indicates that the `plus` function accepts two naturals and returns a natural. The infix notation allows us to write `plus` using the usual infix `+` notation. The `priority := high` bit makes it more likely that uses of `+` will get resolved to our custom `plus` function (for our own natural number type ($\mathbb{N}$) based on the surrounding context).

If we write `zero` as `0` and `suc m` as `1 + m`, the definition turns into two familiar equations:

$$\begin{aligned} 0 + n &= n \\ (1 + m) + n &= 1 + (m + n) \end{aligned}$$

The first follows because zero is an identity for addition, and the second because addition is associative. In its most general form, associativity is written

$$(m + n) + p = m + (n + p)$$

meaning the location of the parentheses is irrelevant.

An example of how we can use these constructors (better more illustrative example second I think):

```
example : 3 + 2 = 5 :=
  calc
    3 + 2
  = .suc (2 + 2)           := rfl
  _ = .suc (.suc (1 + 2))   := rfl
  _ = .suc (.suc (.suc (0 + 2))) := rfl
  _ = .suc (.suc (.suc 2))   := rfl
  _ = 5                     := rfl
```

Note we could've expressed the theorem like so if we wanted to make it more clear in the local context that these literals are formed via our $\mathbb{N}$ type like so: `example : 3 + 2 = (5 :  $\mathbb{N}$ )`

Here's another version closer to the original PLFA text:

```
section
open  $\mathbb{N}$  -- so we don't need dots in front of each ctor
example : 2 + 3 = 5 :=
  calc
    2 + 3
  = (suc (suc zero)) + (suc (suc (suc zero))) := rfl
  _ = suc ((suc zero) + (suc (suc (suc zero)))) := rfl
  _ = suc (suc (zero + (suc (suc (suc zero))))) := rfl
  _ = suc (suc (suc (suc (suc zero)))) := rfl
  _ = 5 := rfl
end
```

Multiplication

Once we have defined addition, we can define multiplication as repeated addition:

```
def mult :  $\mathbb{N}$  ->  $\mathbb{N}$  ->  $\mathbb{N}$ 
| .zero, n    => .zero
| (.suc m), n => n + (mult m n)
```



```
-- infix syntax rules for infix syntax:
syntax:70 (priority := high) term:70 " * " term:71 : term

macro_rules
| `($a * $b) => `(mult $a $b)
```

Computing $m * n$ returns the sum of m copies of n . Again, rewriting turns the definition into two familiar equations:

$$\begin{aligned} 0 * n &= 0 \\ (1 + m) * n &= n + (m * n) \end{aligned}$$

Exercise *-example (practice)

Compute $3 * 4$ writing out your reasoning as a chain of equations using the equations for $*$. You do not need to step through the evaluation of $+$

potential sol:

```
example : 3 * 4 = 12 :=
  calc
    3 * 4
  _ = 4 + (2 * 4)           := rfl
  _ = 4 + (4 + (1 * 4))     := rfl
  _ = 4 + (4 + (4 + (0 * 4))) := rfl
  _ = 4 + (4 + (4 + 0))     := rfl
  _ = 4 + 4 + 4             := rfl
  _ = 12                    := rfl
```

Exercise exponentiation (recommended)

Define exponentiation, which is given by the following equations:

$$\begin{aligned} m ^ 0 &= 1 \\ m ^ (1 + n) &= m * (m ^ n) \end{aligned}$$

potential sol:

```
def exponent : ℕ -> ℕ -> ℕ
| m , 0      => 1
| m , (.suc n) => m * (exponent m n)

syntax:80 (priority := high) term:81 " ^ " term:80 : term

macro_rules
| `($a ^ $b) => `(exponent $a $b)
```

Check that 3^4 is 81:

```
#check (3 ^ 4 = 81)
```

Monus

We can also define subtraction for natural numbers. Since there are no negative natural numbers, if we subtract a larger number from a smaller number we will take the result to be zero. This adaption of subtraction to naturals is called monus (a twist on *minus*).

Monus is our first use of a definition that uses pattern matching against both arguments:

```
def monus : ℕ -> ℕ -> ℕ
| m, .zero          => m
| .zero, (.suc n)    => .zero
| (.suc m), (.suc n) => monus m n

syntax:80 (priority := high) term:81 " ÷ " term:80 : term

macro_rules
| `($a ÷ $b) => `(monus $a $b)
```

where lean performs case analysis to ensure that all the cases are covered.

- consider the second argument:
 - – if it is zero, then the first equation applies
 - – if it is suc n, then consider the first argument
 - – a) if it is zero, then the second equation applies
 - – b) if it is suc m, then the third equation applies

Lean will raise an error if all the cases are not covered. As with addition and multiplication, the recursive definition is well founded because monus on bigger numbers is defined in terms of monus on smaller numbers.

For example, let's subtract two from three:

```
example : 3 ÷ 2 = 1 :=
  calc
    2 ÷ 1
  = 1 ÷ 0 := rfl
_ = 1      := rfl
```

We did not use the second equation at all, but it will be required if we try to subtract a larger number from a smaller one:

```

example : 2 ÷ 3 = 0 :=
  calc
    1 ÷ 2
  _ = 0 ÷ 1      := rfl -- desugars to:
  _ = 0 ÷ (.suc 0) := rfl
  _ = 0          := rfl

```

Currying

We have chosen to represent a function of two arguments in terms of a function of the first argument that returns a function of the second argument. This trick goes by the name *currying*.

Lean4, like other functional languages such as Haskell and ML, is designed to make currying easy to use. Function arrows associate to the right and application associates to the left:

$\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ associates like so: $\mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$

and

`plus 2 3` stands for `(plus 2) 3`